Julia

Task	Command	Notes
Version manager	`juliaup`	`juliaup add 1.11` to install a version
New project	`] generate PkgName`	creates `Project.toml` + `src/`
Add dependency	`] add PkgName`	`] add PkgName@0.5` for version
Install deps	`] instantiate`	installs from `Manifest.toml`
Build	`] build`	runs `deps/build.jl` if present
Run	`julia script.jl`	`--project=.` for local env
REPL	`julia`	`]` pkg, `;` shell, `?` help modes
Test	`] test`	runs `test/runtests.jl`
Format	`JuliaFormatter.format_file()`	`] add JuliaFormatter`
Lint	—	LSP provides diagnostics
LSP	`LanguageServer.jl`	started by nvim and Emacs eglot

Basic Syntax

`function f(x) ... end`	Function
`f(x) = expr`	Short form
`x -> expr`	Anonymous function
`if / elseif / else / end`	Conditional
`x ? a : b`	Ternary
`for x in xs ... end`	For loop
`while cond ... end`	While loop
`begin ... end`	Block expression
`let x = v ... end`	Local scope
`return expr`	Return value

Types & Dispatch

`struct S ... end`	Immutable struct
`mutable struct S`	Mutable struct
`abstract type T end`	Abstract type
`T <: S`	Subtype
`f(x::Int)`	Type annotation
`f(x::T) where T`	Parametric method
`methods(f)`	List methods
`@which f(x)`	Which method?
`isa(x, T)`	Type check
`typeof(x)`	Runtime type
`Union{A, B}`	Union type
`Nothing`	Unit / void type

Common Types

`Int64, Float64`	Numeric defaults
`Bool`	true / false
`String, Char`	Text
`Symbol` (`:name`)	Interned name
`Nothing`	Singleton
`Missing`	Missing data
`Tuple{A,B}`	Fixed tuple
`NamedTuple`	Named tuple
`Dict{K,V}`	Dictionary
`Set{T}`	Set

Arrays

`[1, 2, 3]`	Vector (1-indexed)
`[1 2; 3 4]`	Matrix
`zeros(m, n)`	Zero matrix
`ones(m, n)`	Ones matrix
`fill(v, m, n)`	Fill value
`a[i]`	Index (1-based)
`a[2:5]`	Slice
`a[end]`	Last element
`push!(a, x)`	Append (mutate)
`length(a)`	Length
`size(a)`	Dimensions
`a'`	Adjoint (transpose)
`vcat(a, b) / hcat(a, b)`	Stack vert / horiz
`vec(a)`	Flatten to vector

Comprehensions & Generators

`[f(x) for x in xs]`	Array comprehension
`[f(x) for x in xs if p]`	Filtered
`[f(i,j) for i=1:m, j=1:n]`	2D comprehension
`Dict(k => v for ...)`	Dict comprehension
`(f(x) for x in xs)`	Generator (lazy)

Strings & I/O

`"$x and $(expr)"`	String interpolation
`string(a, b)`	Concatenate
`join(xs, ",")`	Join with sep
`split(s, ",")`	Split string
`strip(s)`	Trim whitespace
`println(x)`	Print with newline
`@show x`	Debug print
`@info "msg" x y`	Logging
`open(f, "r") do io`	File handle
`read(f, String)`	Read file
`write(f, data)`	Write file

Functional Patterns

`map(f, xs)`	Apply f to each
`filter(p, xs)`	Keep matching
`reduce(f, xs)`	Fold
`mapreduce(f, op, xs)`	Map then reduce
`any(p, xs) / all(p, xs)`	Existential / universal
`zip(xs, ys)`	Pair elements
`enumerate(xs)`	Index + element
`sort(xs; by=f)`	Sort by projection
`unique(xs)`	Unique values
`xs .|> f`	Broadcast pipe
`x |> f |> g`	Pipeline
`f ∘ g`	Compose (unicode)
`do ... end`	Closure block

Broadcasting

`f.(xs)`	Broadcast call
`a .+ b`	Elementwise ops
`@. expr`	Broadcast everything
`a .== b`	Elementwise compare

Error Handling

`try ... catch e ... end`	Try/catch
`finally ... end`	Always runs
`throw(ErrorException(m))`	Throw
`error("msg")`	Shorthand throw
`@assert cond "msg"`	Assertion

Macros & Metaprogramming

`@time expr`	Benchmark
`@benchmark expr`	BenchmarkTools
`@code_warntype f(x)`	Type stability
`@macroexpand @m x`	Expand macro
`macro m(x) ... end`	Define macro
`:(expr)`	Quote expression
`$(expr)`	Interpolate in quote

Julia — Scientific Computing

LinearAlgebra

`using LinearAlgebra`	Import
`A * B`	Matrix multiply
`A \ b`	Solve Ax = b
`det(A)`	Determinant
`inv(A)`	Inverse
`eigen(A)`	Eigen-decomposition
`svd(A)`	SVD
`qr(A)`	QR factorization
`norm(x)`	Vector/matrix norm
`dot(a, b)` / `a ⋅ b`	Dot product
`cross(a, b)`	Cross product
`tr(A)`	Trace
`rank(A)`	Rank
`I`	Identity operator
`Diagonal(v)`	Diagonal matrix
`Symmetric(A)`	Symmetric view

Statistics

`using Statistics`	Import
`mean(x)`	Mean
`median(x)`	Median
`std(x) / var(x)`	Std dev / variance
`cor(x, y)`	Correlation
`quantile(x, p)`	Quantile

Distributions.jl

`Normal(`μ`, `σ`)`	Normal distribution
`Uniform(a, b)`	Uniform distribution
`Poisson(`λ`)`	Poisson distribution
`rand(d, n)`	Sample n values
`pdf(d, x) / cdf(d, x)`	PDF / CDF
`fit(Normal, data)`	Fit distribution

DataFrames.jl

`using DataFrames, CSV`	Import
`CSV.read(f, DataFrame)`	Read CSV
`CSV.write(f, df)`	Write CSV
`DataFrame(a=xs, b=ys)`	Create DataFrame
`df.col` / `df[:, :col]`	Column access
`df[1:5, :]`	Row slice
`filter(:col => p, df)`	Filter rows
`select(df, :a, :b)`	Select columns
`transform(df, :a => f)`	Add/modify column
`groupby(df, :key)`	Group by
`combine(gd, :v => mean)`	Aggregate
`innerjoin(a, b; on=:k)`	Join tables
`sort(df, :col)`	Sort
`describe(df)`	Summary stats
`dropmissing(df)`	Drop missing rows

Plots.jl / Makie.jl

`using Plots`	Import (GR default)
`plot(x, y)`	Line plot
`scatter(x, y)`	Scatter plot
`histogram(x)`	Histogram
`heatmap(z)`	Heatmap
`plot!(x, y)`	Add to current plot
`xlabel!("x")`	Set label
`title!("T")`	Set title
`savefig("f.png")`	Save figure
`using CairoMakie`	Makie backend
`fig, ax, plt = lines(x,y)`	Makie plot

DifferentialEquations.jl

`using DifferentialEquations`	Import
`f(u, p, t) = ...`	ODE function
`ODEProblem(f, u0, tspan)`	Define problem
`solve(prob)`	Solve (auto alg)
`solve(prob, Tsit5())`	Specific solver
`sol(t)`	Interpolate solution
`plot(sol)`	Plot solution

Optimization (Optim.jl)

`using Optim`	Import
`optimize(f, x0)`	Minimize f
`optimize(f, lo, hi)`	Bounded 1D
`NelderMead(), BFGS()`	Algorithms
`Optim.minimizer(res)`	Optimal x
`Optim.minimum(res)`	Optimal f(x)

Pkg Operations

`] status`	List packages
`] update`	Update all
`] remove PkgName`	Remove package
`] activate .`	Activate local env
`] activate --temp`	Temp environment
`] dev PkgName`	Dev mode (editable)
`] pin PkgName`	Pin version
`] gc`	Garbage collect

Key Packages

`DataFrames`	Tabular data
`CSV`	CSV read/write
`Plots / Makie`	Visualization
`DifferentialEquations`	ODE/SDE/PDE
`Optim / JuMP`	Optimization
`Flux / Lux`	Neural networks
`Distributions`	Probability
`StatsBase`	Statistics
`HTTP`	HTTP client
`JSON3`	JSON parsing